

CITYCARE HOSPITAL MANAGEMENT SYSTEM

WEEK 4: PATIENT MEDICINE ORDER MANAGEMENT SYSTEM

1. SCENARIO

CityCare Hospital needs a database system to manage patient medicine orders. Patients can receive multiple medicines, and each order should maintain details such as order date, medicines purchased, quantity, price, total order amount, and order status.

The system helps the hospital maintain patient order information and manage medicine-related transactions efficiently.

The main functions include:

- Maintaining patient details
- Creating medicine orders for patients
- Adding multiple medicines to an order
- Maintaining quantity and price
- Calculating the total order amount
- Updating order status
- Viewing complete order details
- Generating patient and medicine-wise reports

2. DESIGN PATIENT_ORDERS AND ORDER_DETAILS TABLES

2.1 Patient Table

The Patient table stores the basic information of patients registered in CityCare Hospital.

Attributes

Attribute	Description
Patient_ID	Unique patient identifier
Patient_Name	Name of the patient
Age	Age of the patient
Gender	Gender of the patient
Phone_Number	Patient contact number

2.2 Patient_Orders Table

The Patient_Orders table stores the overall information about a patient's medicine order.

Attributes

Attribute	Description
Order_ID	Unique order identifier
Patient_ID	Patient reference
Order_Date	Date of order placement
Total_Amount	Total bill amount
Order_Status	Pending, Delivered or Cancelled

```

SQL> CREATE TABLE Patient_Orders (
2     Order_ID NUMBER PRIMARY KEY,
3     Patient_ID NUMBER NOT NULL,
4     Order_Date DATE DEFAULT SYSDATE,
5     Total_Amount NUMBER(10,2) CHECK (Total_Amount >= 0),
6     Order_Status VARCHAR2(20) DEFAULT 'Pending',
7     CONSTRAINT fk_order_patient
8         FOREIGN KEY (Patient_ID)
9         REFERENCES Patient(Patient_ID),
10    CONSTRAINT chk_order_status
11        CHECK (Order_Status IN ('Pending', 'Delivered', 'Cancelled'))
12 );

```

Table created.

```
SQL> DESC Patient_Orders;
```

Name	Null?	Type
ORDER_ID	NOT NULL	NUMBER
PATIENT_ID	NOT NULL	NUMBER
ORDER_DATE		DATE
TOTAL_AMOUNT		NUMBER(10,2)
ORDER_STATUS		VARCHAR2(20)

3. ORDER_DETAILS TABLE

The Order_Details table stores the individual medicines included in each patient order.

Attributes

Attribute	Description
Order_Detail_ID	Unique order detail identifier
Order_ID	Order reference
Medicine_ID	Medicine reference
Quantity	Number of medicines ordered
Price	Price of the medicine

```

SQL> CREATE TABLE Order_Details (
2     Order_Detail_ID NUMBER PRIMARY KEY,
3     Order_ID NUMBER NOT NULL,
4     Medicine_ID NUMBER NOT NULL,
5     Quantity NUMBER CHECK (Quantity > 0),
6     Price NUMBER(10,2) CHECK (Price > 0),
7     CONSTRAINT fk_detail_order
8         FOREIGN KEY (Order_ID)
9         REFERENCES Patient_Orders(Order_ID),
10    CONSTRAINT fk_detail_medicine
11        FOREIGN KEY (Medicine_ID)
12        REFERENCES Medicine(Medicine_ID)
13 );

```

Table created.

```
SQL> DESC Order_Details;
```

Name	Null?	Type
ORDER_DETAIL_ID	NOT NULL	NUMBER
ORDER_ID	NOT NULL	NUMBER
MEDICINE_ID	NOT NULL	NUMBER
QUANTITY		NUMBER
PRICE		NUMBER(10,2)

```
SQL> |
```

4. ESTABLISH RELATIONSHIPS

4.1 Patient and Patient_Orders

One patient can place multiple medicine orders.

Each order belongs to one patient.

Relationship:

Patient (1) ----- (Many) Patient_Orders

The Patient_ID in Patient_Orders is a foreign key referencing the Patient table.

4.2 Patient_Orders and Medicine

One patient order can contain multiple medicines.

One medicine can appear in multiple patient orders.

Relationship:

Patient_Orders (Many) ----- (Many) Medicine

This many-to-many relationship is implemented using the Order_Details table.

The Order_Details table acts as the bridge between Patient_Orders and Medicine.

5. APPLY CONSTRAINTS

The following constraints are used in the Week 4 database:

Primary Key

Primary keys uniquely identify each record.

Patient_ID → Patient

Order_ID → Patient_Orders

Order_Detail_ID → Order_Details

Medicine_ID → Medicine

Foreign Key

Foreign keys establish relationships between tables.

Patient_Orders.Patient_ID

→ Patient.Patient_ID Order_Details.Order_ID

→ Patient_Orders.Order_ID Order_Details.Medicine_ID

→ Medicine.Medicine_ID

NOT NULL

Important fields cannot contain NULL values.

Patient_Name
Patient_ID
Order_ID
Medicine_ID
Quantity

CHECK

CHECK constraints ensure valid values.

CHECK (Age > 0)CHECK (Quantity > 0)CHECK (Price > 0)CHECK
(Total_Amount >= 0)

The order status is restricted to:

Pending
Delivered
Cancelled

6. MANAGE PATIENT MEDICINE ORDERS

The system performs the following operations:

1. Create a new order for a patient.
2. Add multiple medicines to an order.
3. Calculate the total order amount.
4. Update order status.
5. Update medicine quantity.
6. Delete cancelled orders.
7. View complete order details.

7. INSERT PATIENT DETAILS

```
SQL> INSERT INTO Patient VALUES
  2 (301, 'Ananya Kumar', 25, 'Female', '9876500011');

1 row created.

SQL>
SQL> INSERT INTO Patient VALUES
  2 (302, 'Rahul Sharma', 40, 'Male', '9876500012');

1 row created.

SQL>
SQL> INSERT INTO Patient VALUES
  2 (303, 'Meena Devi', 32, 'Female', '9876500013');

1 row created.

SQL>
SQL> INSERT INTO Patient VALUES
  2 (304, 'Arun Raj', 55, 'Male', '9876500014');

1 row created.

SQL>
SQL> INSERT INTO Patient VALUES
  2 (305, 'Kavitha S', 28, 'Female', '9876500015');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Patient;
```

```
SQL> SELECT * FROM Patient;

PATIENT_ID
-----
PATIENT_NAME
-----
      AGE  GENDER  PHONE_NUMBER
-----
      301
Ananya Kumar
      25 Female    9876500011

      302
Rahul Sharma
      40 Male      9876500012

PATIENT_ID
-----
PATIENT_NAME
-----
      AGE  GENDER  PHONE_NUMBER
-----

      303
Meena Devi
      32 Female    9876500013

      304
Arun Raj

PATIENT_ID
-----
PATIENT_NAME
-----
      AGE  GENDER  PHONE_NUMBER
-----

      55 Male      9876500014

      305
Kavitha S
      28 Female    9876500015
```

8. CREATE NEW PATIENT ORDERS

```
SQL> INSERT INTO Patient_Orders
  2 (Order_ID, Patient_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (501, 301, SYSDATE, 0, 'Pending');

1 row created.

SQL>
SQL> INSERT INTO Patient_Orders
  2 (Order_ID, Patient_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (502, 302, SYSDATE, 0, 'Pending');

1 row created.

SQL>
SQL> INSERT INTO Patient_Orders
  2 (Order_ID, Patient_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (503, 303, SYSDATE, 0, 'Pending');

1 row created.

SQL>
SQL> INSERT INTO Patient_Orders
  2 (Order_ID, Patient_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (504, 304, SYSDATE, 0, 'Pending');

1 row created.

SQL>
SQL> INSERT INTO Patient_Orders
  2 (Order_ID, Patient_ID, Order_Date, Total_Amount, Order_Status)
  3 VALUES
  4 (505, 305, SYSDATE, 0, 'Pending');

1 row created.
```

```
SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Patient_Orders;
```

ORDER_ID	PATIENT_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
501	301	04-OCT-26	0	Pending
502	302	04-OCT-26	0	Pending
503	303	04-OCT-26	0	Pending
504	304	04-OCT-26	0	Pending
505	305	04-OCT-26	0	Pending

9. ADD MULTIPLE MEDICINES TO ORDERS

```

SQL> INSERT INTO Order_Details
  2 VALUES (1, 501, 201, 2, 2.50);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 VALUES (2, 501, 202, 1, 8.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 VALUES (3, 502, 203, 3, 3.50);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 VALUES (4, 503, 204, 2, 5.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 VALUES (5, 504, 205, 1, 12.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2 VALUES (6, 505, 206, 3, 4.00);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

```

```

SQL>
SQL> SELECT * FROM Order_Details;

```

ORDER_DETAIL_ID	ORDER_ID	MEDICINE_ID	QUANTITY	PRICE
1	501	201	2	2.5
2	501	202	1	8
3	502	203	3	3.5
4	503	204	2	5
5	504	205	1	12
6	505	206	3	4

```

6 rows selected.

```

10. CALCULATE TOTAL ORDER AMOUNT

The total amount is calculated using:

```
SQL> UPDATE Patient_Orders po
  2  SET Total_Amount = (
  3      SELECT SUM(od.Quantity * od.Price)
  4      FROM Order_Details od
  5      WHERE od.Order_ID = po.Order_ID
  6  );

5 rows updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Patient_Orders
  2  ORDER BY Order_ID;
```

ORDER_ID	PATIENT_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
501	301	04-OCT-26	13	Pending
502	302	04-OCT-26	10.5	Pending
503	303	04-OCT-26	10	Pending
504	304	04-OCT-26	12	Pending
505	305	04-OCT-26	12	Pending

11. UPDATE OPERATION

The order status can be updated when the medicine order is delivered.

For example, changing Order 501 from Pending to Delivered:

```

505          305 04-OCT-26          12 Pending
SQL> UPDATE Patient_Orders
2 SET Order_Status = 'Delivered'
3 WHERE Order_ID = 501;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Patient_Orders
2 WHERE Order_ID = 501;

ORDER_ID PATIENT_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-----
501          301 04-OCT-26          13 Delivered

SQL> |

```

12. UPDATE MEDICINE QUANTITY

The quantity of a medicine can also be modified.

```

SQL>
SQL> SELECT * FROM Patient_Orders
2 WHERE Order_ID = 501;

ORDER_ID PATIENT_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-----
501          301 04-OCT-26          13 Delivered

SQL> UPDATE Order_Details
2 SET Quantity = 5
3 WHERE Order_Detail_ID = 3;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Order_Details
2 WHERE Order_Detail_ID = 3;

ORDER_DETAIL_ID ORDER_ID MEDICINE_ID QUANTITY PRICE
-----
3          502          203          5          3.5

```

13. DELETE OPERATION

Cancelled orders can be removed from the system.

First, change Order 505 to Cancelled:

```
505      305 04-OCT-26      12 Pending

SQL> UPDATE Patient_Orders
  2  SET Order_Status = 'Delivered'
  3  WHERE Order_ID = 501;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Patient_Orders
  2  WHERE Order_ID = 501;

ORDER_ID PATIENT_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-----
501      301 04-OCT-26      13 Delivered

SQL> |
```

```
SQL> DELETE FROM Order_Details
  2  WHERE Order_ID = 505;

1 row deleted.

SQL> DELETE FROM Patient_Orders
  2  WHERE Order_ID = 505;

1 row deleted.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Patient_Orders
  2  ORDER BY Order_ID;

ORDER_ID PATIENT_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-----
501      301 04-OCT-26      13 Delivered
502      302 04-OCT-26     10.5 Pending
503      303 04-OCT-26      10 Pending
504      304 04-OCT-26      12 Pending
```

14. REPORT 1: PATIENT ORDER HISTORY

The Patient Order History report displays:

- Patient Name
- Order ID
- Order Date
- Total Amount
- Order Status

```
SQL> SELECT
  2     p.Patient_Name,
  3     po.Order_ID,
  4     po.Order_Date,
  5     po.Total_Amount,
  6     po.Order_Status
  7 FROM Patient p
  8 JOIN Patient_Orders po
  9     ON p.Patient_ID = po.Patient_ID
 10 ORDER BY po.Order_ID;
```

PATIENT_NAME

ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS

Ananya Kumar
501 04-OCT-26 13 Delivered

Rahul Sharma
502 04-OCT-26 10.5 Pending

Meena Devi
503 04-OCT-26 10 Pending

PATIENT_NAME

ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS

Arun Raj
504 04-OCT-26 12 Pending

5. REPORT 2: MEDICINE-WISE ORDER REPORT

This report displays:

- Medicine Name
- Number of times ordered
- Total quantity ordered

```
SQL> SELECT
  2     m.Medicine_Name,
  3     COUNT(od.Order_ID) AS Times_Ordered,
  4     SUM(od.Quantity) AS Total_Quantity
  5 FROM Medicine m
  6 JOIN Order_Details od
  7     ON m.Medicine_ID = od.Medicine_ID
  8 GROUP BY m.Medicine_Name
  9 ORDER BY m.Medicine_Name;
```

MEDICINE_NAME

TIMES_ORDERED TOTAL_QUANTITY

Amoxicillin

1

1

Azithromycin

1

1

Cetirizine

1

5

MEDICINE_NAME

TIMES_ORDERED TOTAL_QUANTITY

Omeprazole

1

2

Paracetamol

1

2

16. REPORT 3: PATIENT PURCHASE ANALYSIS

This report analyses the purchasing activity of each patient.

It displays:

- Patient Name
- Total Orders
- Total Purchase Amount
- Average Order Amount

```
SQL> SELECT
  2     p.Patient_Name,
  3     COUNT(po.Order_ID) AS Total_Orders,
  4     NVL(SUM(po.Total_Amount), 0) AS Total_Purchase_Amount,
  5     NVL(AVG(po.Total_Amount), 0) AS Average_Order_Amount
  6 FROM Patient p
  7 LEFT JOIN Patient_Orders po
  8     ON p.Patient_ID = po.Patient_ID
  9 GROUP BY p.Patient_Name
 10 ORDER BY Total_Purchase_Amount DESC;
```

PATIENT_NAME

TOTAL_ORDERS	TOTAL_PURCHASE_AMOUNT	AVERAGE_ORDER_AMOUNT
--------------	-----------------------	----------------------

Ananya Kumar		
1	13	13

Arun Raj		
1	12	12

Rahul Sharma		
1	10.5	10.5

PATIENT_NAME

TOTAL_ORDERS	TOTAL_PURCHASE_AMOUNT	AVERAGE_ORDER_AMOUNT
--------------	-----------------------	----------------------

Meena Devi		
1	10	10

Kavitha S		
0	0	0

17. SUMMARY

In Week 4, the CityCare Hospital Management System was extended to manage patient medicine orders.

The Patient_Orders table stores overall order information, while the Order_Details table stores individual medicines included in each order.

Primary keys, foreign keys, NOT NULL constraints and CHECK constraints were implemented to maintain data integrity.

The system supports creating orders, adding multiple medicines, calculating total order amounts, updating order information, deleting cancelled orders and generating useful reports.

18. CONCLUSION

The Week 4 implementation provides an organized database structure for managing patient medicine orders in CityCare Hospital.

The system makes it possible to maintain patient orders, medicine quantities, prices and order status efficiently. The generated reports provide useful information about patient order history, medicine usage and patient purchase analysis.